SYMBIOSIS INSTITUTE OF TECHNOLOGY, HYDERABAD
Constituent of Symbiosis International (Deemed University), Pune
(Established under Section 3 of the UGC Act of 1956 vide notification number F.9-12/2001-U.3 of the Government of India)
Re-Accredited by NAAC with 'A++' Grade

FAKE AND REAL NEWS CLASSIFICATION\nUSING MACHINE LEARNING

A supervised learning approach using TF-IDF and text classification algorithms

Name	A. Tejkiran	PRN	24070721005
Branch	Computer Science and Engineering (CSE)	Assessment	Supervised Machine Learning
Submission date	12 November 2026	Dataset	Fake and Real News Dataset

Abstract

The project develops a machine learning system to classify news articles as fake or real by analysing their textual content. The Fake and Real News dataset was downloaded from Kaggle and contains two separate collections, which were assigned binary labels: 0 for fake news and 1 for real news. The records were combined, shuffled and cleaned by removing missing values and duplicate rows, resulting in 44,689 articles. The title and article body were joined, text was normalised, and TF-IDF was used to convert the text into a numerical feature representation with a maximum of 5,000 features. The data was divided into training and testing sets using an 80:20 stratified split. Four classifiers—Logistic Regression, Decision Tree, Random Forest and K-Nearest Neighbours—were trained and evaluated. Random Forest achieved the highest test accuracy of 99.83%, followed by Decision Tree (99.49%), Logistic Regression (98.90%) and KNN (71.65%). The Random Forest model was selected as the final model and its confusion matrix showed only 15 incorrect predictions among 8,938 test articles. The trained model and TF-IDF vectorizer were saved for later use.

Keywords: Fake news detection, supervised learning, TF-IDF, text classification, Random Forest.

1. Introduction

Online news is shared rapidly through websites and social media. Along with genuine reports, misleading or fabricated articles can also circulate and influence readers. Manually checking every article is time-consuming. Machine learning can assist by learning language patterns from labelled examples and predicting whether a new article is likely to be fake or real.

1.1 Problem statement

The objective is to build a supervised binary classification model that accepts a news title and article text and predicts one of two classes: Fake News (0) or Real News (1). The project uses the text content as input and compares multiple classification algorithms on the same held-out test set.

2. Dataset Description and Exploratory Analysis

2.1 Dataset source and structure

The notebook downloads the "Fake and Real News Dataset" from Kaggle. It reads the files Fake.csv and True.csv separately. A label column is added to each dataset: fake articles receive label 0 and real articles receive label 1. The two dataframes are concatenated and shuffled with random_state = 42 to mix the classes reproducibly.

Field	Description	Use in this project
title	Headline of the news article	Combined with article body
text	Main body of the article	Primary text input
subject	News subject/category	Retained in dataset; not used as model input
date	Article date	Retained in dataset; not used as model input
label	0 = Fake, 1 = Real	Target class

The combined dataset initially contained 44,898 rows and five columns. The initial inspection reported no missing values and 209 duplicate rows. After dropping missing values and duplicates, the cleaned dataset contained 44,689 records.

2.2 Class distribution

The notebook visualises the distribution of the two target labels using a count plot. The graph indicates that both fake and real news classes have substantial representation. The classification task is therefore treated as a binary supervised learning problem.

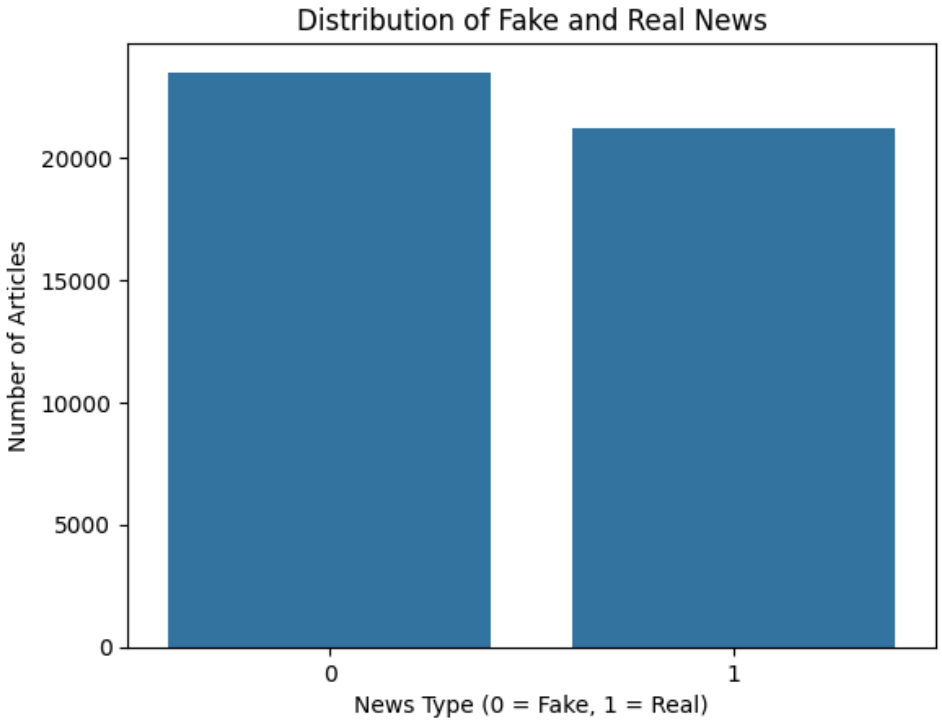


Figure 1. Output generated by the notebook showing the distribution of fake (0) and real (1) news articles.

2.3 Data cleaning

The following cleaning operations are applied: (i) remove rows containing missing values with dropna(); (ii) remove duplicate rows with drop_duplicates(); and (iii) reset the dataframe index. The final dataset shape printed by the notebook is (44689, 5).

2.4 Text preparation

A new content column is created by joining the title and text fields. The clean_text() function converts text to lowercase, removes web links, removes HTML tags, removes characters other than English letters and whitespace, and collapses repeated spaces. The cleaned text is then used for model training.

3. Methodology and Model Development

3.1 Train–test split

The cleaned content column is selected as the feature (X), while label is selected as the target (y). The train_test_split() function uses test_size = 0.2, random_state = 42 and stratify = y. This produces 35,751 training articles and 8,938 testing articles. Stratification preserves the approximate class proportions in both subsets.

Subset	Number of articles	share
Training set	35,751	80%
Testing set	8,938	20%
Total	44,689	100%

3.2 TF-IDF feature extraction

Machine learning algorithms require numerical input. The notebook uses TfidfVectorizer with max_features = 5000 and stop_words = "english". TF-IDF (Term Frequency–Inverse Document Frequency) represents words according to their importance in a document relative to the collection. The vectorizer is fitted on the training text and then applied to the test text. The resulting feature matrices have shapes (35751, 5000) for training and (8938, 5000) for testing.

3.3 Classification algorithms

Algorithm	Role in the experiment
Logistic Regression	Linear classifier used to estimate the probability of fake or real news. max_iter is set to 1000.
Decision Tree	Tree-based classifier that learns decision rules from the TF-IDF features. random_state = 42.
Random Forest	Ensemble of 100 decision trees; predictions are combined to improve generalisation.
K-Nearest Neighbours	Classifies an article using the labels of its five nearest training examples (k = 5).

3.4 Evaluation measures

The notebook calculates accuracy, precision, recall and F1-score for each model. Accuracy measures the proportion of all correct predictions. Precision measures how many articles predicted as real are actually real (positive class = 1). Recall measures how many actual real articles are identified. F1-score is the harmonic mean of precision and recall. The best model is selected by sorting the results in descending order of F1-score.

3.5 Implementation workflow

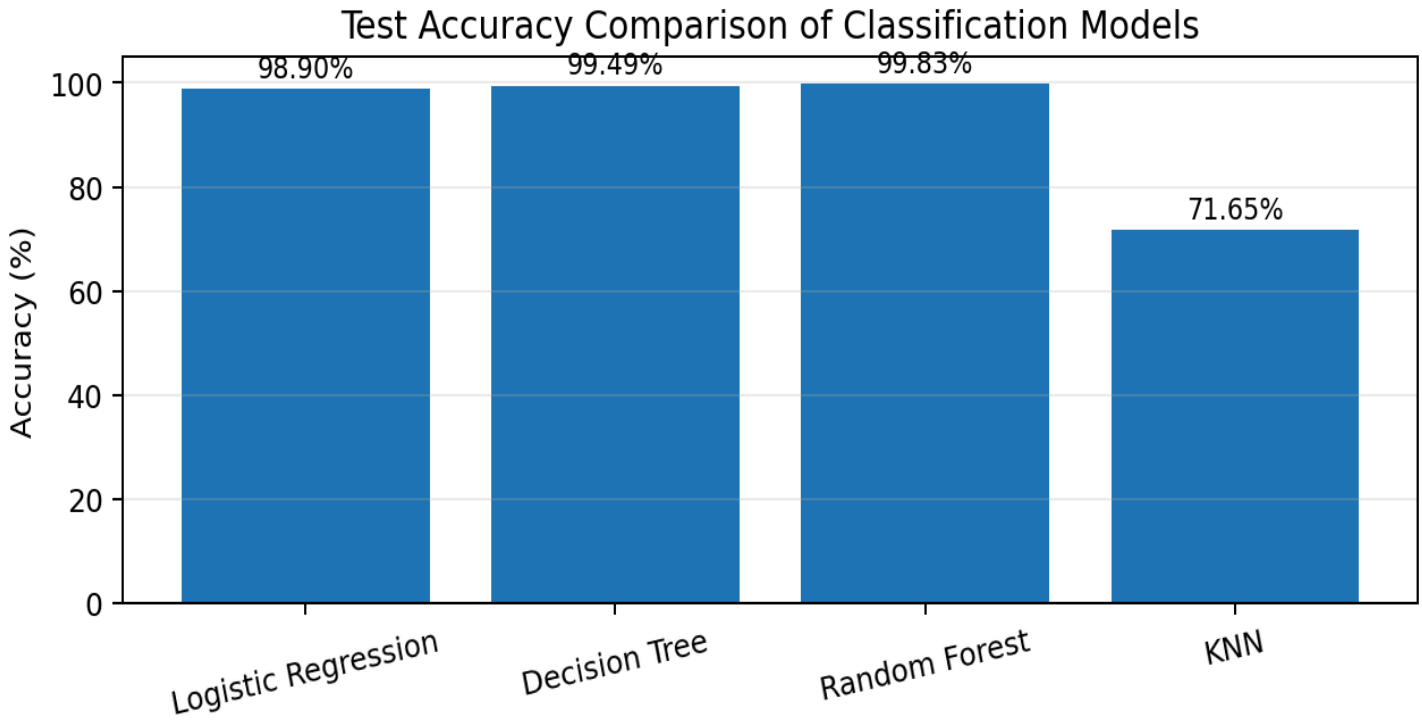
Dataset download → CSV loading → binary labelling → concatenation and shuffle → missing-value and duplicate removal → title/body combination → text cleaning → stratified split → TF-IDF vectorisation → model training → evaluation → best-model selection → saving the model.

4. Experimental Results and Comparison

All four models were trained using the same training features and evaluated on the same test set of 8,938 articles. The following values are taken from the results dataframe printed by the notebook.

Model	Accuracy	Precision	Recall	F1-score
Random Forest	99.83%	99.76%	99.88%	99.82%
Decision Tree	99.49%	99.57%	99.34%	99.46%
Logistic Regression	98.90%	98.48%	99.22%	98.85%
KNN (k = 5)	71.65%	93.53%	43.26%	59.16%

Figure 2. Comparison of test accuracy for the four machine learning models.



4.1 Result interpretation

Random Forest achieved the best overall result with 99.83% accuracy and an F1-score of 0.9982. Decision Tree ranked second with 99.49% accuracy, while Logistic Regression achieved 98.90%. KNN achieved 71.65% accuracy and a lower recall of 43.26%, indicating that it missed a considerable number of actual real-news articles under the selected settings.

The results table shows that Random Forest offers the strongest balance of precision and recall among the tested models. Its F1-score is the highest, so the notebook selects Random Forest as the final classifier.

4.2 Selected model

The selection code sorts the results dataframe by F1 Score in descending order and retrieves the model name from the first row. The output confirms: "Best Model: Random Forest". The corresponding prediction array is then used to generate the classification report and confusion matrix.

5. Final Model Evaluation and Prediction

5.1 Confusion matrix

The confusion matrix evaluates the Random Forest model on the 8,938 test articles. Label 0 represents fake news and label 1 represents real news. Rows indicate actual labels and columns indicate predicted labels.

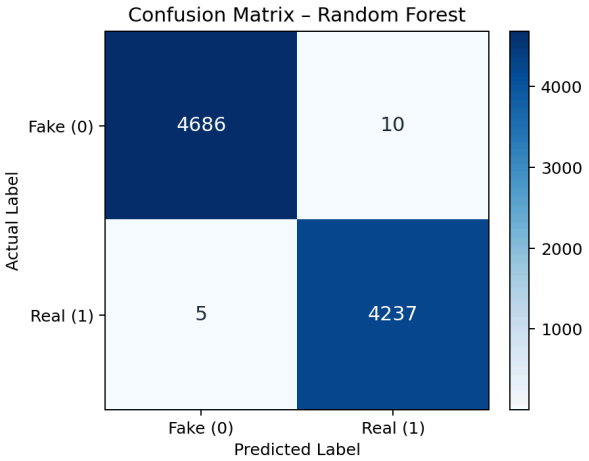


Figure 3. Confusion matrix of the selected Random Forest classifier.

Actual class	Predicted fake (0)	Predicted real (1)	Total
Fake (0)	4,686	10	4,696
Real (1)	5	4,237	4,242
Total	4,691	4,247	8,938

The model correctly classified 4,686 fake articles and 4,237 real articles. It incorrectly labelled 10 fake articles as real and 5 real articles as fake. Thus, 15 out of 8,938 test examples were misclassified.

5.2 Classification report

The notebook's classification_report() output is rounded to two decimal places and displays precision, recall and F1-score as 1.00 for both classes. The underlying confusion matrix shows the small number of errors above; therefore, the rounded report should not be interpreted as mathematically perfect classification.

Class	Precision (approx.)	Recall (approx.)	F1-score (approx.)	Support
Fake news (0)	0.9989	0.9979	0.9984	4,696
Real news (1)	0.9976	0.9988	0.9982	4,242

5.3 Prediction on a sample input

The notebook defines predict_news(news), which cleans a supplied news string, transforms it with the fitted TF-IDF vectorizer and sends it to the selected Random Forest model. The sample sentence used is: "The government announced a new policy today." The recorded output is "Prediction: FAKE NEWS".

5.4 Model saving

The final model and fitted TF-IDF vectorizer are stored together in a dictionary and exported using joblib.dump() to the file fake_news_detection_model.pkl. The notebook confirms "Model saved successfully!". This file can be loaded in a later Python program to make predictions without retraining the model.

6. Conclusion, Limitations and References

6.1 Conclusion

This project demonstrates a supervised machine learning workflow for distinguishing fake news from real news using article text. The dataset was cleaned and prepared by combining headline and body text, normalising the text and converting it into TF-IDF features. Four algorithms were compared on a stratified 80:20 train–test split. Random Forest achieved the best performance, with 99.83% accuracy and an F1-score of approximately 99.82%, and was selected as the final model. The confusion matrix records 15 errors among 8,938 test articles. The model and vectorizer were also saved for future prediction.

6.2 Limitations

• The dataset contains news from a particular collection and time period; performance on current or different news sources may vary.
• The model learns statistical patterns in the training text and does not independently verify facts, sources or claims.
• The notebook uses a single random train–test split. Additional validation on an independent dataset would provide stronger evidence of generalisation.
• The KNN result is substantially lower than the other models, and further parameter tuning or alternative text representations could be explored.

6.3 Future enhancements

Future work may include testing on more recent and diverse news articles, using cross-validation, tuning model hyperparameters, comparing additional algorithms such as Linear SVM or Naive Bayes, and building a simple web interface where a user can paste a news article and receive a predicted label. Human fact-checking should remain important for high-impact decisions.

References

[1] Kaggle, "Fake and Real News Dataset," dataset by clmentbisaillon. Available at: https://www.kaggle.com/datasets/clmentbisaillon/fake-and-real-news-dataset.

[2] F. Pedregosa et al., "Scikit-learn: Machine Learning in Python," Journal of Machine Learning Research, vol. 12, pp. 2825–2830, 2011.

[3] Scikit-learn documentation, "TfidfVectorizer," "LogisticRegression," "DecisionTreeClassifier," "RandomForestClassifier," and "KNeighborsClassifier." https://scikit-learn.org/

Project outcome: Random Forest was selected as the best-performing model in the submitted notebook, with a test accuracy of 99.83%.